- *UI state variables*—State stored as variables or as an object: `{isLoading: true}`
- *Client-side interactivity*—For example, a dropdown menu that shows or hides options based on user selection or a button that updates a counter when clicked

Importantly, UI state is confined to the client and cannot be directly transferred back to the server. Its primary role is to reflect changes in the AI state, ensuring that the user interface accurately represents the current application state.

How to set up the UI and AI state

The Vercel AI SDK provides a set of helpers to facilitate the setup and management of both UI and AI states. These helpers are designed to work seamlessly with React's state management best practices while implementing the principles outlined in our state flow diagram.

4.2.2 Key components for UI state management

Let's explore the essential functions and hooks provided by the Vercel AI SDK that make this possible. There are five key functions you should know: `createAI`, `getMutableAIState`, `useActions`, `useUIState`, and `useAIState`.

createAI

`createAI` is a function that creates an AI context provider. It's the foundation of our state management system, allowing us to define

- *Initial AI and UI states*—For example, the initial AI state might include `userProfile: { name: '', email: '' }` or `currentTask: null`.
- *Actions that can be performed* (typically server actions)—These actions are operations that interact with the server, such as `fetchUserData`, which retrieves user information.
- *Types for both AI and UI states*—Types define the shape and structure of the states. For instance, `AIState` might include types like `UserProfile` or `Task`, while `UIState` could define types like `SidebarState` or `ThemeSettings`.

This function sets up the structure for managing and synchronizing states across your application. You create this function, and then it returns to a React context component, which you can place on the root layout page. Then you will gain access to the server actions and `useState` hooks to manage both UI and AI state.

Say you are building a web application where users can manage their profiles and settings. At the start, the application needs to load the user's profile and settings from the server and set up the initial UI state.

When the app starts, `createAI` initializes with an initial AI state. It sets `userProfile` to have an empty name and email, set to default values (light theme and notifications enabled). It defines server actions like `fetchUserData` and `updateUserSettings`. The `AIProvider` wraps the root layout of the app, making the AI and UI state accessible throughout the application. This setup ensures that any component within `AppLayout` can access and manage these states and perform actions like fetching data or updating settings. The following listing shows the example reflected as code.